

Designing a programming language for local
reasoning and easy debugging

Whoami

- Robin Heggelund Hansen
- Consultant at Bekk
- Creator of Gren

Gren

- Is a purely functional, statically typed programming language with ML-syntax
- It aims to be small and easy to learn, while still being performant and expressive enough for general use
- Gren targets JavaScript, for maximum portability
- Still in early development

Why?

There is no such thing as a perfect
programming language ...

... but you can get pretty close within a specific domain

The sort of projects I work on

- Big corporations in public and private sectors
- Not technically advanced (rest services)
- Relatively low-traffic
- Important services that can make the news when there are issues

An interesting thing about consultants

- Don't tend to stick around
- I tend to work in codebases I have little experience with

My ideal programming language

- Make it easy to understand what code does and doesn't do, without requiring that I know the entire codebase
- Has guarantees and tooling that make it easy to pinpoint and fix problems
- Performance isn't terribly important, but it shouldn't get in my way

My ideal programming language

Enables local reasoning, and easy debugging

Local reasoning

What does that mean?

Error handling

```
canViewAccount : Request -> Account -> Bool
canViewAccount req account =
    let
        userDetails =
            decodeUserDetails req
    in
    case userDetails of
        Admin _ ->
            True

        User details ->
            List.member account.id details.accountIds
```

Managing side-effects

```
loadFromCache :: Key -> IO (Maybe Value)
```

Managing side-effects

```
module FileSystem (..)
```

```
openForRead : Permission -> String -> Task AccessError (ReadableFileHandle a)
```


Tradeoffs

- Code size
- But to me, that is a tradeoff worth making

Debugging

When local reasoning won't do

Step-Debuggers are useful

- Makes for easy exploration of the running application
- Makes it easier to learn how the language works
- Sometimes, reasoning fails

Challenges of lazy evaluation

```
encodeHelp :: Int -> String -> String
encodeHelp num acc =
    let clamped =
        num .&. 31

        newNum =
            num `Bit.shiftR` 5

        newClamped =
            if newNum > 0
            then clamped .|. 32
            else clamped

        newAcc =
            base64Table ! newClamped : acc
    in if newNum > 0
        then encodeHelp newNum newAcc
        else List.reverse newAcc
```

Challenges of lazy evaluation

```
encodeHelp :: Int -> String -> String
encodeHelp num acc =
    let clamped =
        num .&. 31

        newNum =
            num `Bit.shiftR` 5

        newClamped =
            if newNum > 0
            then clamped .|. 32
            else clamped

        newAcc =
            base64Table ! newClamped : acc
    in if newNum > 0 --< BREAK HERE
        then encodeHelp newNum newAcc
        else List.reverse newAcc
```

Challenges of lazy evaluation

```
encodeHelp :: Int -> String -> String
encodeHelp num acc =
    let clamped =
        num .&. 31

        newNum =
            num `Bit.shiftR` 5 --< STEPS TO HERE

        newClamped =
            if newNum > 0
            then clamped .|. 32
            else clamped

        newAcc =
            base64Table ! newClamped : acc
    in if newNum > 0
        then encodeHelp newNum newAcc
        else List.reverse newAcc
```

Challenges of lazy evaluation

```
encodeHelp :: Int -> String -> String
encodeHelp num acc =
    let clamped =
        num .&. 31

        newNum =
            num `Bit.shiftR` 5

        newClamped =
            if newNum > 0
            then clamped .|. 32
            else clamped

        newAcc =
            base64Table ! newClamped : acc
    in if newNum > 0
        then encodeHelp newNum newAcc
        else List.reverse newAcc
```

Challenges of lazy evaluation

```
encodeHelp :: Int -> String -> String
encodeHelp num acc =
    let clamped =
        num .&. 31

        newNum =
            num `Bit.shiftR` 5

        newClamped =
            if newNum > 0
            then clamped .|. 32
            else clamped

        newAcc =
            base64Table ! newClamped : acc
    in if newNum > 0
        then encodeHelp newNum newAcc
        else List.reverse newAcc --< NOW HERE
```


Importance of stack traces

Tradeoffs

- Strict evaluation makes it easier to step through the code
- Readable stack traces make it easy to locate grivious errors
- Being able to debug the actual source code complicates and slows down the compiler
- Using the target platform's primitive types makes it easier to inspect state

Do we need a new language for this?

Problems with new languages

- Learning them takes time and commitment
- People usually have limited time to learn new things
- Few are willing to bet on a language without a future
- To be successful the language needs to be small, and have a low complexity budget.
- Also, should be portable.

Why make a new language?

- Haskell is big and complex. Could do better on local reasoning and debugging.
- Elm is great! ... but it's hard to use for backends or terminal applications. Also, debugging experience could be better.

Gren

- Small, has simple but powerful features that compose, and aims to be learnable with a low time investment
- Great for local reasoning
- Integrates well with the JS debugger
- Can use it almost everywhere

Questions?

Gren: <https://gren-lang.org>

Mastodon: @robinheghan@snabelen.no